解決策の検証とユースケース
NetApp Solutions

NetApp
June 07, 2022

This PDF was generated from https://docs.netapp.com/ja-jp/netapp-solutions/containers/rh-os-n_use_case_pipeline.html on June 07, 2022. Always check docs.netapp.com for the latest.

目次

解決策の検証とユースケース：ネットアップを使用した Red Hat OpenShift 1
永続的ストレージを使用した Jenkins CI / CD パイプラインの導入：ネットアップでの Red Hat OpenShift 1
NetApp ONTAP を使用して Red Hat OpenShift にマルチテナンシーを設定します 11

解決策の検証とユースケース：ネットアップを使用した Red Hat OpenShift

このページに記載する例は、ネットアップでの Red Hat OpenShift の解決策の検証と使用事例です。

- "永続的ストレージを使用した Jenkins CI/CD パイプラインの導入"
- "ネットアップを使用して Red Hat OpenShift でマルチテナンシーを構成します"
- "NetApp ONTAP を使用した Red Hat OpenShift Virtualization"
- "ネットアップを使用した Red Hat OpenShift での Kubernetes 向けの高度なクラスタ管理"

"次は、ビデオとデモです"

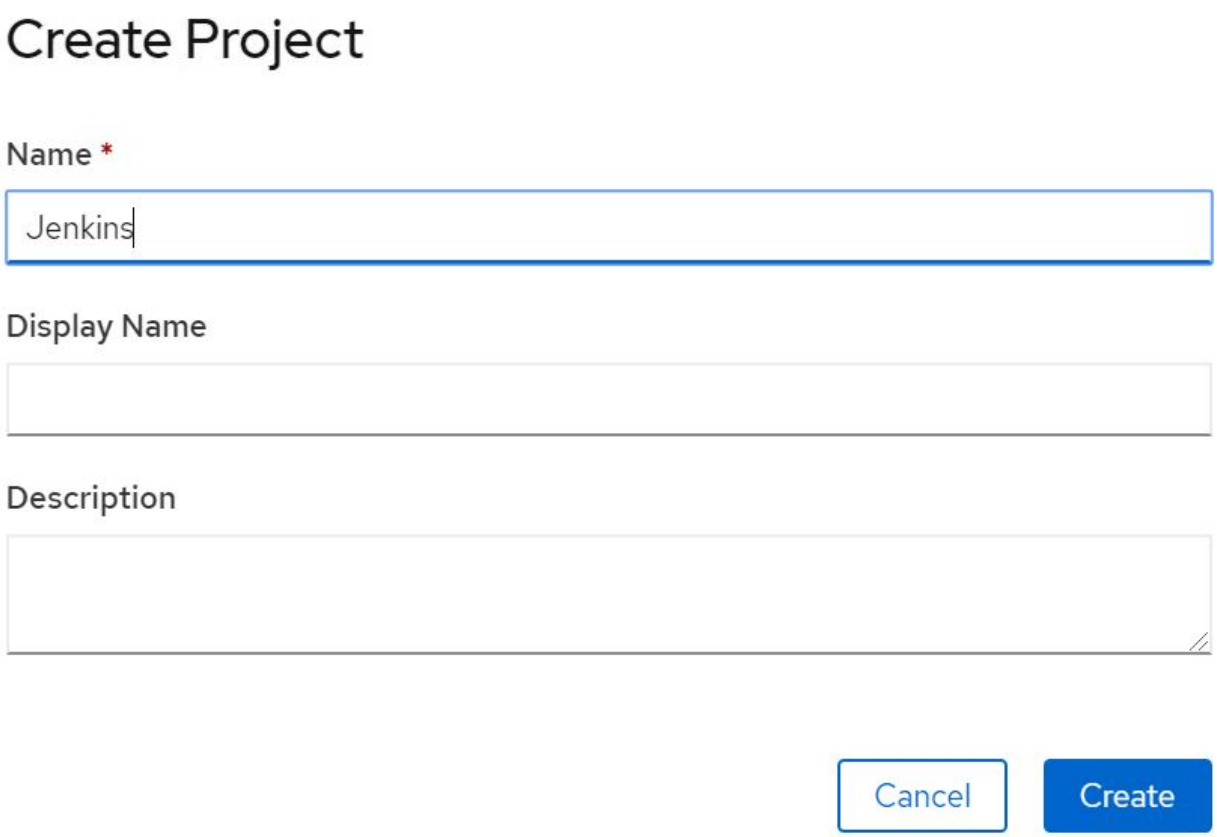
永続的ストレージを使用した Jenkins CI / CD パイプラインの導入：ネットアップでの Red Hat OpenShift

このセクションでは、 Jenkins との継続的統合 / 継続的配信または導入（ CI / CD ）パイプラインを導入して解決策の動作を検証する手順について説明します。

Jenkins の導入に必要なリソースを作成します

Jenkins アプリケーションの導入に必要なリソースを作成するには、次の手順に従います。

1. Jenkins という名前の新しいプロジェクトを作成します。



2. この例では、永続的ストレージを使用して Jenkins を導入しています。Jenkins ビルドをサポートするには、 PVC を作成します。[ストレージ] > [永続的ボリューム要求] の順に選択し、 [永続的ボリューム要求の作成] をクリックします。作成したストレージクラスを選択し、永続ボリューム要求名が Jenkins であることを確認し、適切なサイズとアクセスモードを選択して、作成をクリックします。

Project: jenkins

Create Persistent Volume Claim

Edit YAML

Storage Class

SC basic

Storage class for the new claim.

Persistent Volume Claim Name *

jenkins

A unique name for the storage claim within the project.

Access Mode *

Single User (RWO) Shared Access (RWX) Read Only (ROX)

Permissions to the mounted drive.

Size *

100 GiB

Desired storage capacity.

Use label selectors to request storage

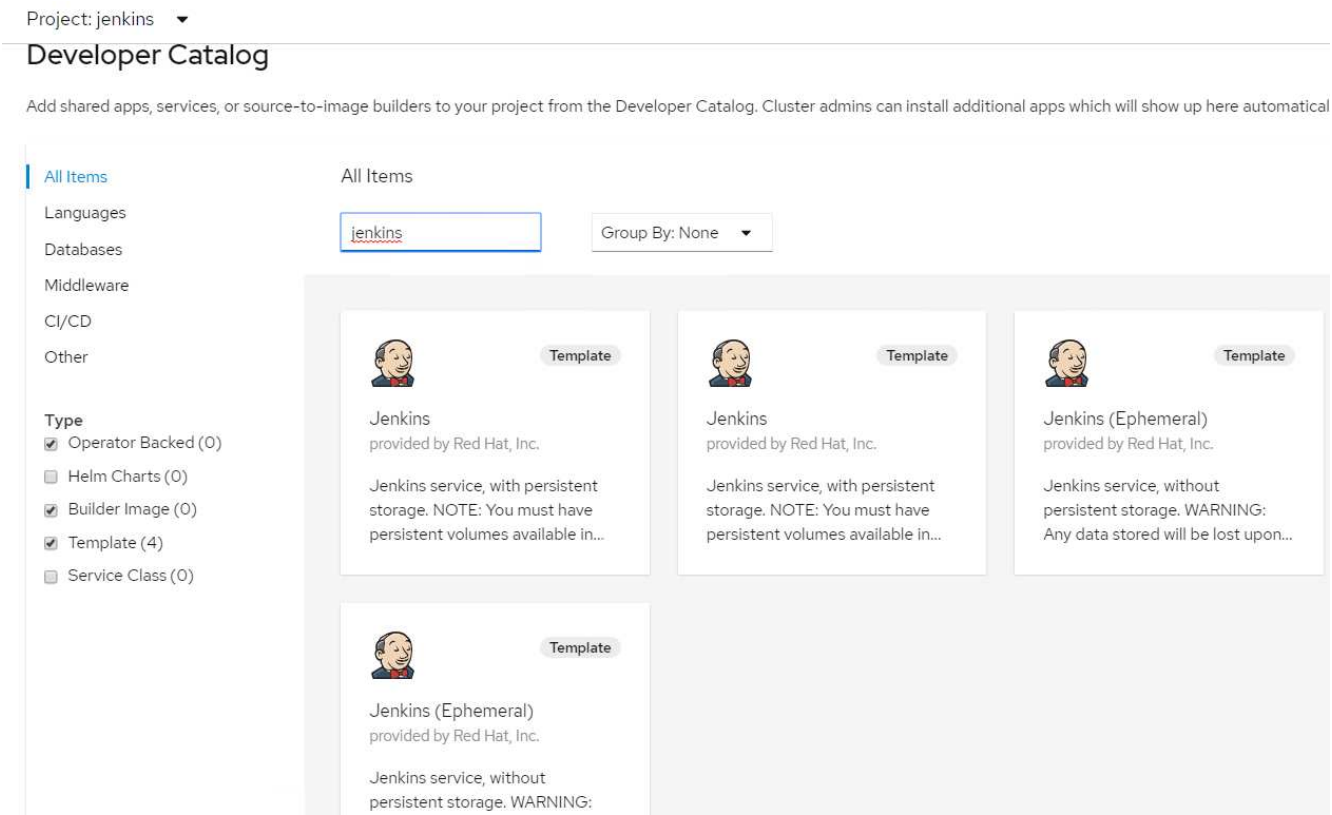
Use label selectors to define how storage is created.

Create Cancel

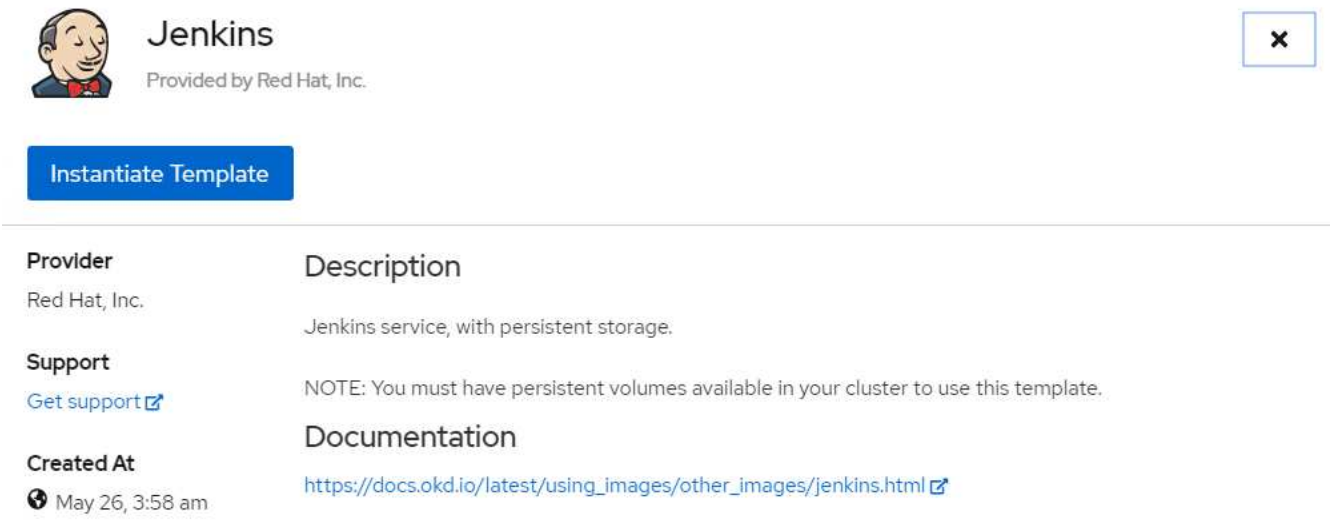
永続的ストレージを使用して **Jenkins** を導入する

永続ストレージを使用して Jenkins を導入するには、次の手順を実行します。

1. 左上隅で、ロールを Administrator から Developer に変更します。+ 追加をクリックし、カタログからを選択します。キーワードでフィルターバーで Jenkins を検索します。永続的ストレージを使用する Jenkins Service を選択します。

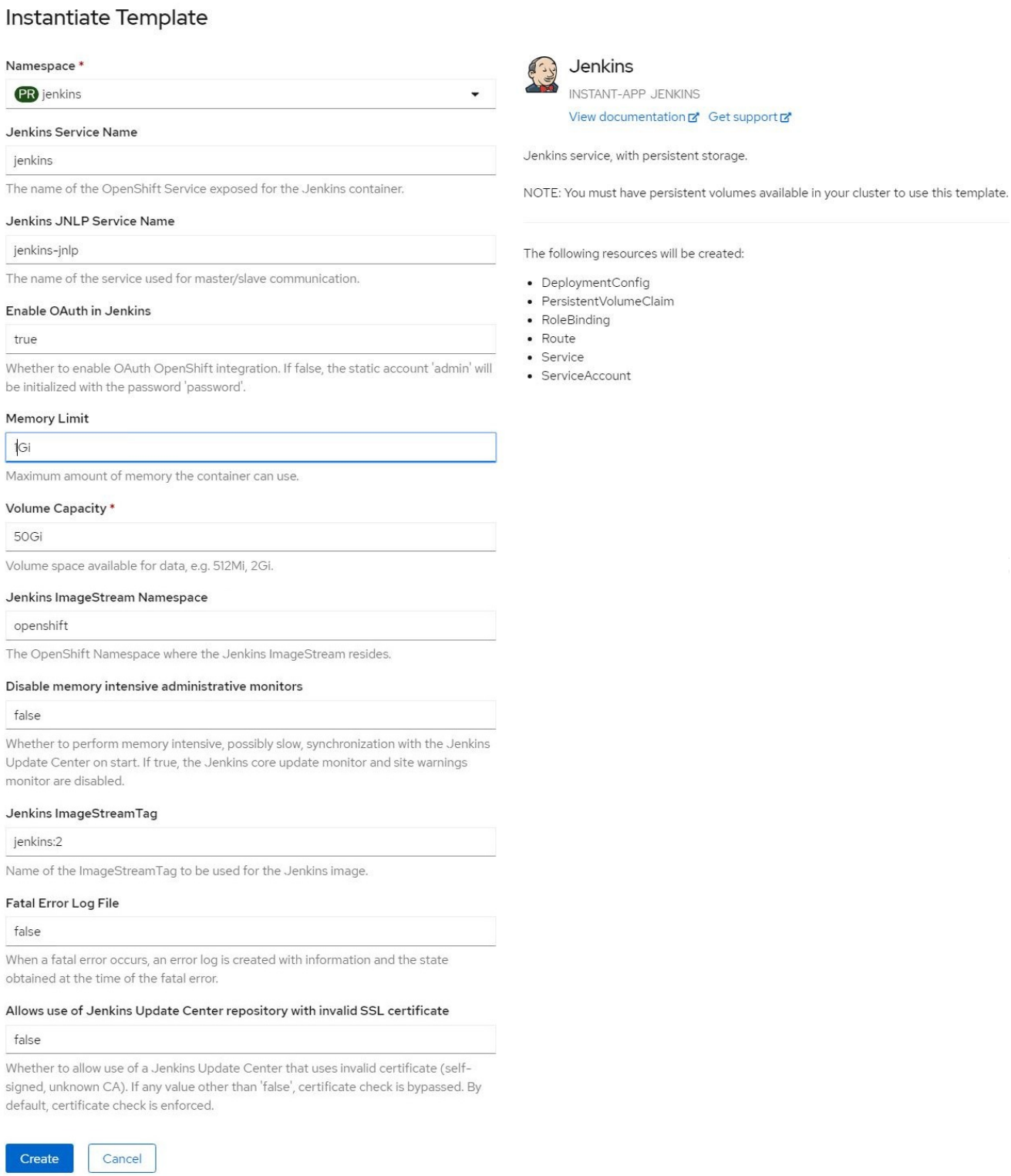


2. 「テンプレートをインスタンス化」をクリックします。

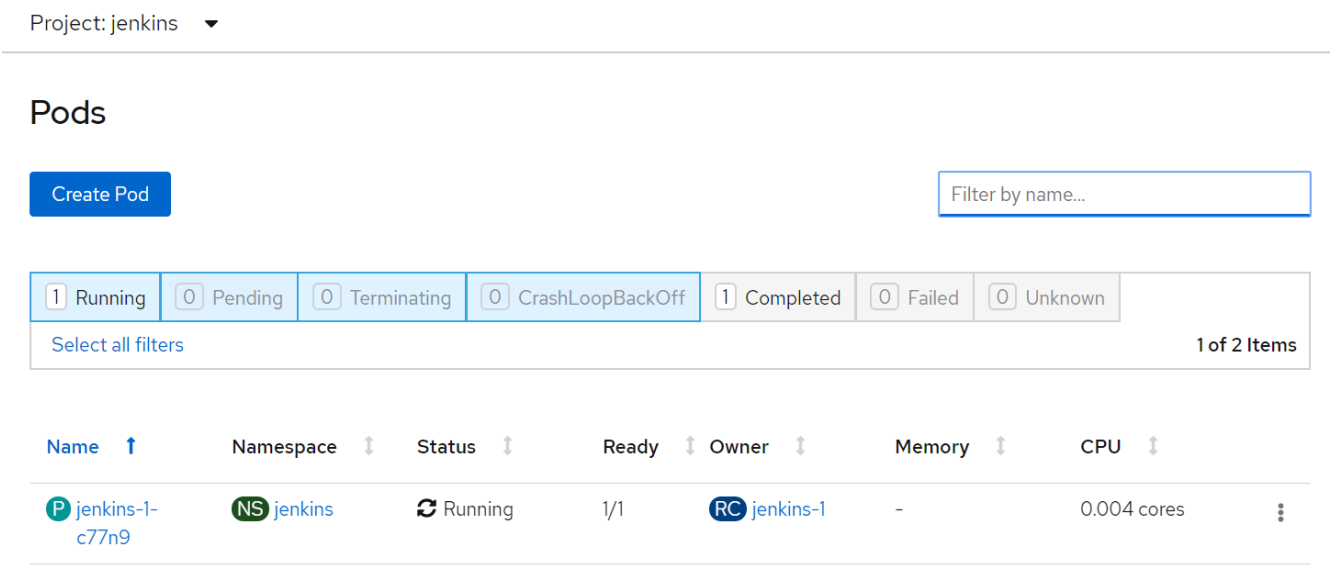


3. デフォルトでは、 Jenkins アプリケーションの詳細が入力されます。要件に基づいてパラメータを変更し、 [作成（ Create ）] をクリックします。このプロセスでは、 OpenShift で Jenkins をサポートするた

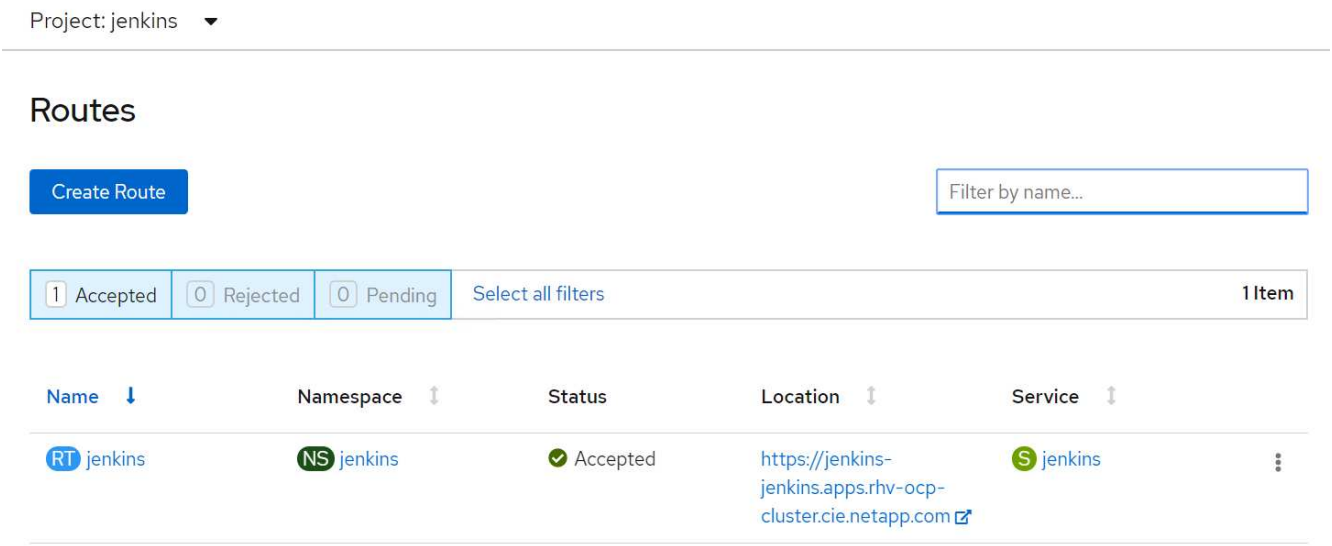
めに必要なリソースがすべて作成されます。



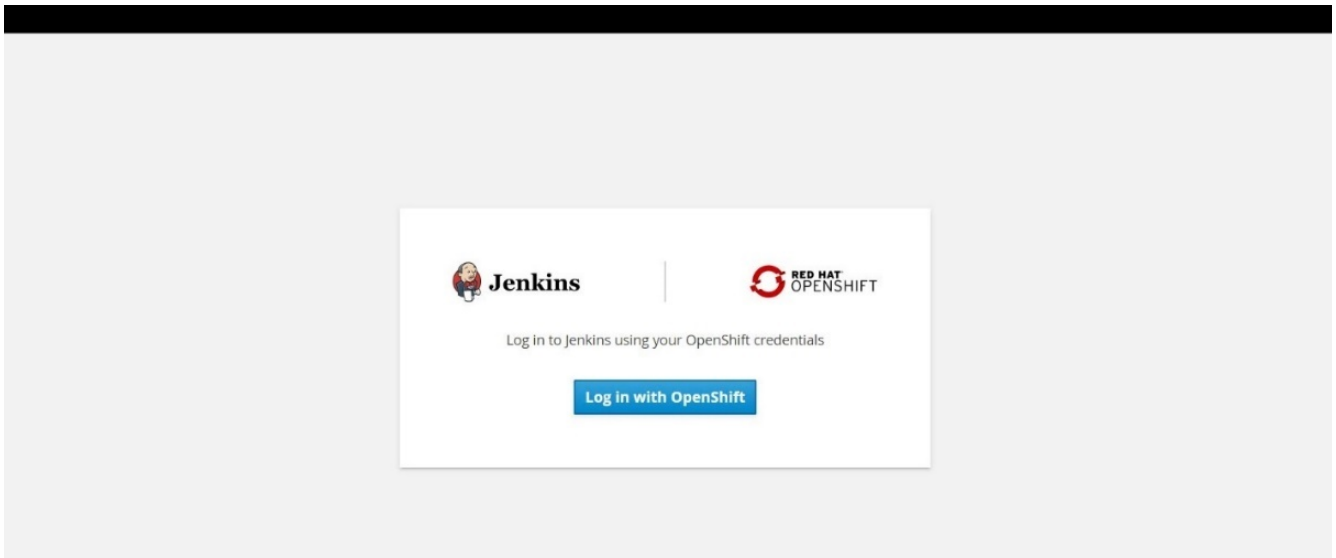
4. Jenkins ポッドが「 Ready 」状態になるまでに約 10 〜 12 分かかります。



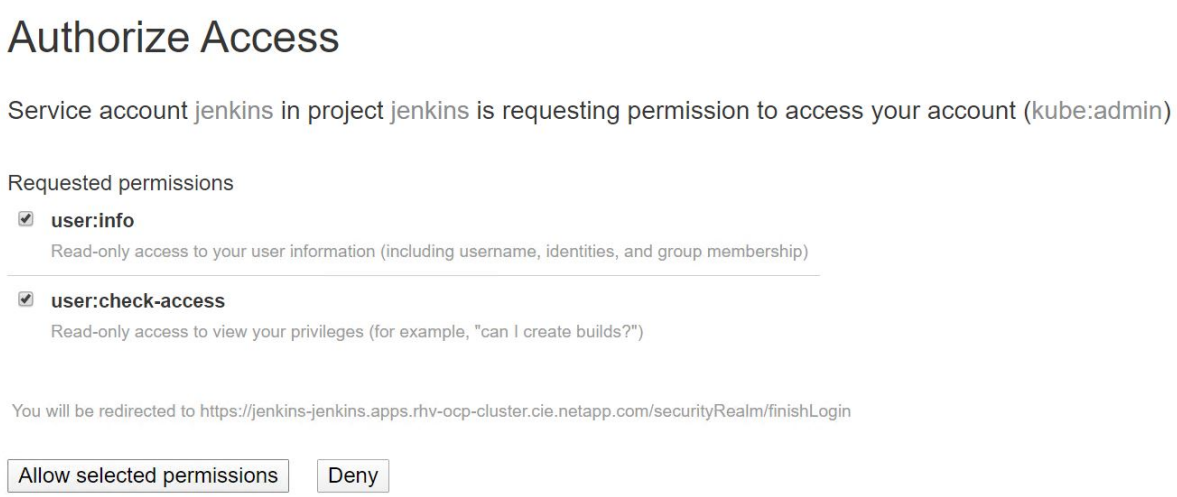
5. ポッドがインスタンス化されたら、ネットワーキング > ルートと進みます。Jenkins の Web ページを開くには、 Jenkins ルート用の URL をクリックします。



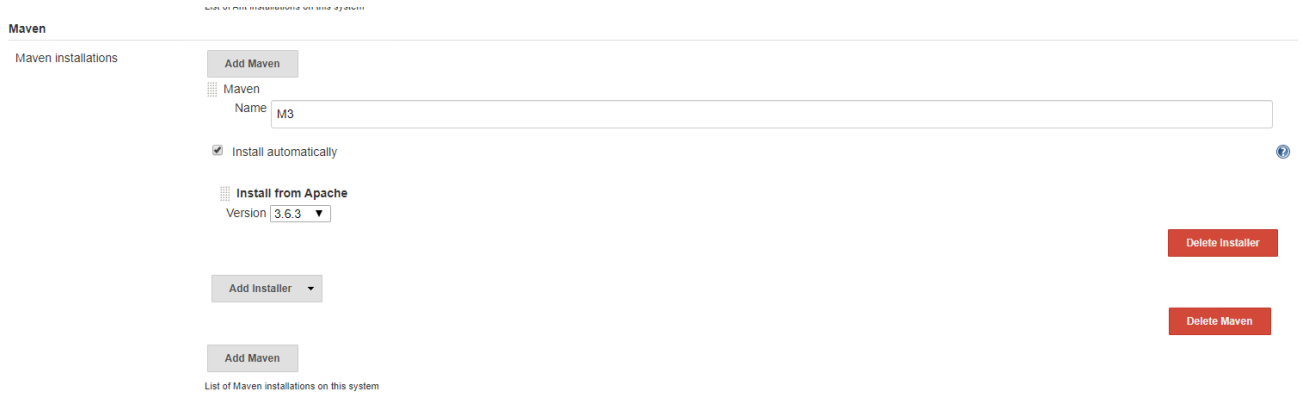
6. Jenkins アプリケーションの作成時に OpenShift OAuth が使用されていたため、「 OpenShift でログイン」をクリックします。



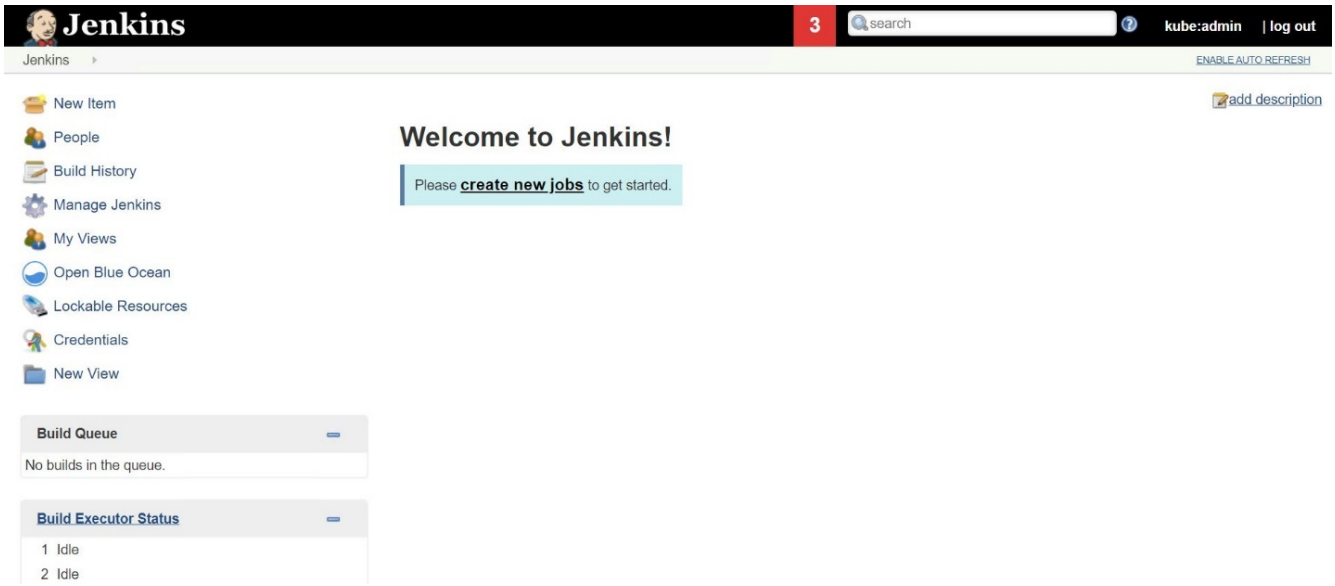
7. Jenkins サービスアカウントに OpenShift ユーザへのアクセスを許可します。



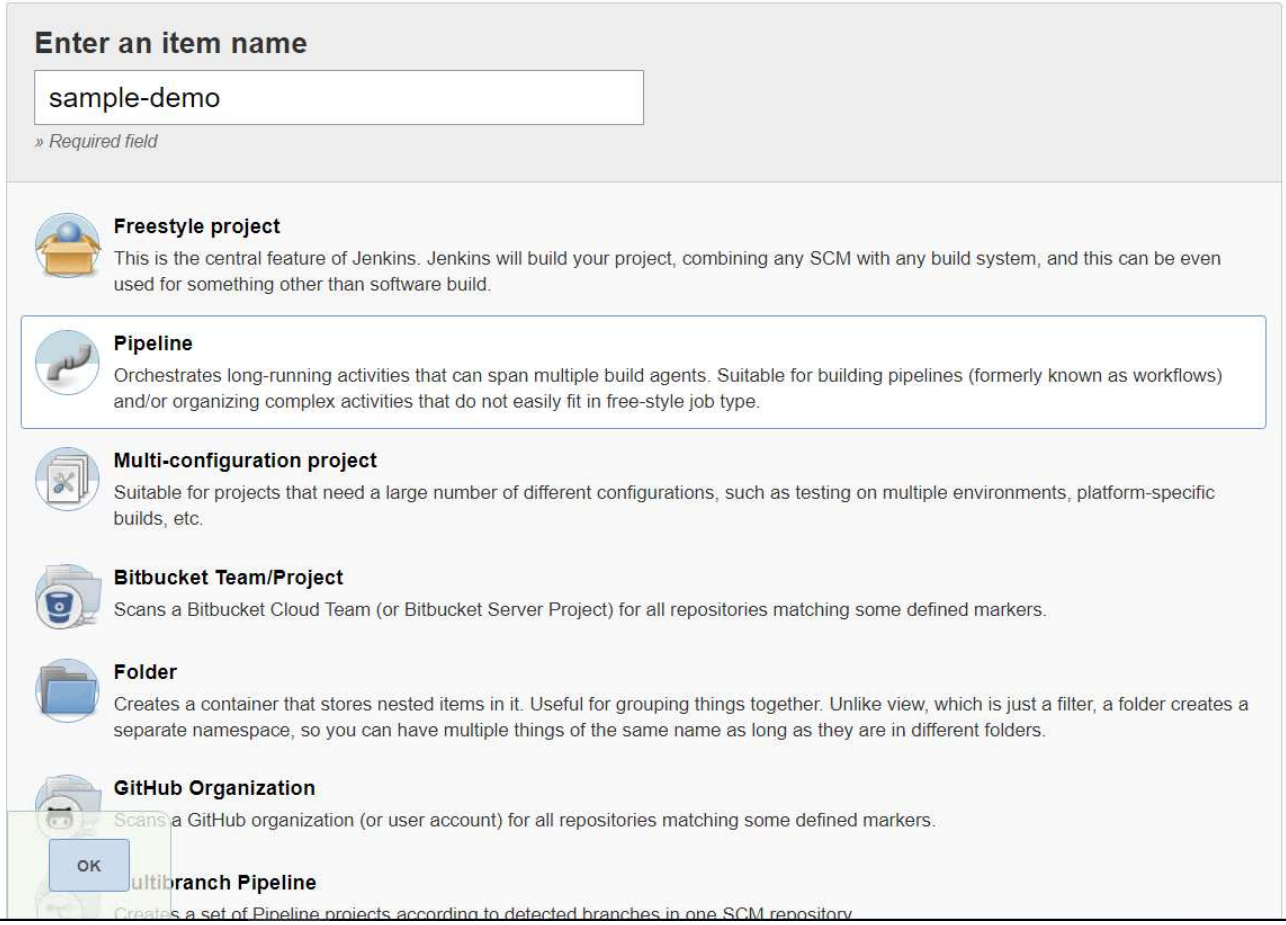
8. Jenkins のようこそページが表示されます。Maven ビルドを使用しているので、まず Maven のインストールを完了します。Manage Jenkins > Global Tool Configuration に移動し、 Maven サブヘッドで Add Maven をクリックします。任意の名前を入力し、 [自動的にインストール] オプションが選択されていることを確認します。[保存] をクリックします .



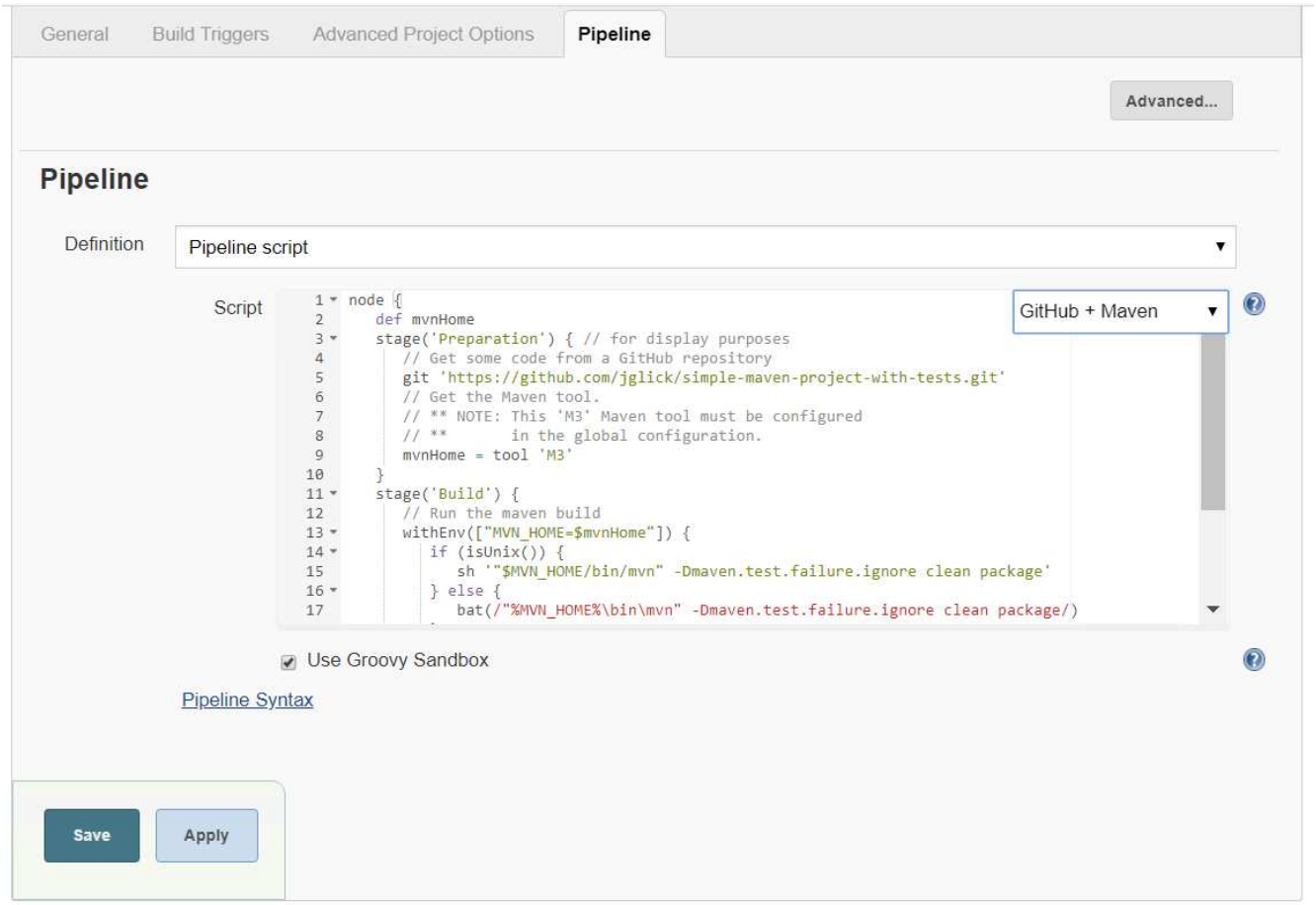
9. CI / CD のワークフローを示すパイプラインを作成できるようになりました。ホームページで、左側のメニューから [新規ジョブの作成] または [新規アイテム] をクリックします。



10. [項目の作成] ページで、任意の名前を入力し、 [パイプライン] を選択して、 [OK] をクリックします。



11. パイプライン (Pipeline) タブを選択しますサンプルパイプラインを試すドロップダウンメニューから、Github + Maven を選択します。コードが自動的に入力されます。[保存] をクリックします .



12. 「今すぐビルド」をクリックして、準備、ビルド、テストの各フェーズで開発を開始します。ビルドプロセス全体が完了してビルドの結果が表示されるまでに数分かかることがあります。

13. コードが変更された場合は、必ずパイプラインを再構築して新しいバージョンのソフトウェアにパッチを適用することで、継続的な統合と継続的な提供を実現できます。[最近の変更] をクリックして、前のバージョンからの変更を追跡します。

"次は、ビデオとデモです"

NetApp ONTAP を使用して Red Hat OpenShift にマルチテナンシーを設定します

ネットアップを使用した Red Hat OpenShift でのマルチテナンシーの構成

コンテナで複数のアプリケーションやワークロードを実行する多くの組織は、アプリケーションやワークロードごとに 1 つの Red Hat OpenShift クラスタを導入する傾向にあります。これにより、アプリケーションやワークロードを厳密に分離し、パフォーマンスを最適化し、セキュリティの脆弱性を軽減できます。ただし、アプリケーションごとに独立した Red Hat OpenShift クラスタを導入するには、独自の問題が発生します。これにより、各クラスタを個別に監視および管理する必要がある運用上のオーバーヘッドが増大し、さまざまなアプリケーションに専用リソースを使用することでコストが増大し、効率的な拡張性が妨げられます。

この問題を解決するには、すべてのアプリケーションまたはワークロードを 1 つの Red Hat OpenShift クラスタで実行することを検討します。しかし、このようなアーキテクチャでは、リソースの分離とアプリケーションセキュリティの脆弱性が大きな課題の 1 つとなっています。あるワークロードのセキュリティの脆弱性は、自然に別のワークロードにオーバーフローする可能性があるため、影響ゾーンが増加します。また、あるアプリケーションによる突然の制御されないリソース使用率は、デフォルトではリソース割り当てポリシーがないため、別のアプリケーションのパフォーマンスに影響を与える可能性があります。

そのため、組織は、たとえば、すべてのワークロードを単一のクラスタで実行しながら、各ワークロードに専用のクラスタのメリットを提供することで、両方の世界で最も優れたソリューションを見つけることができます。

このように効果的な解決策の 1 つは、 Red Hat OpenShift でマルチテナンシーを構成することです。マルチテナンシーは、複数のテナントを同じクラスタ上に共存させ、リソースやセキュリティなどを適切に分離できるアーキテクチャです。この場合、テナントは、特定のユーザグループが専用として使用するように設定されたクラスタリソースのサブセットとみなすことができます。Red Hat OpenShift クラスタでマルチテナンシーを設定する利点は次のとおりです。

- クラスタリソースを許可することで設備投資と運用コストを削減 を共有します
- 運用と管理のオーバーヘッドを軽減
- セキュリティ侵害のクロスコンタミネーションからワークロードを保護
- リソースの競合による予期しないパフォーマンスの低下からワークロードを保護

マルチテナント OpenShift クラスタを完全に実現するには、コンピューティング、ストレージ、ネットワーク、セキュリティなど、異なるリソースバケットに属するクラスタリソースにクォータと制限を設定する必要があります。この解決策のすべてのリソースバケットの特定の側面について説明しますが、 ネットアップでは、 NetApp ONTAP を基盤とする Astra Trident によって動的に割り当てられるストレージリソースにマルチテナンシーを設定することで、複数のワークロードで提供または消費されるデータを分離し、保護するためのベストプラクティスに焦点を当てています。

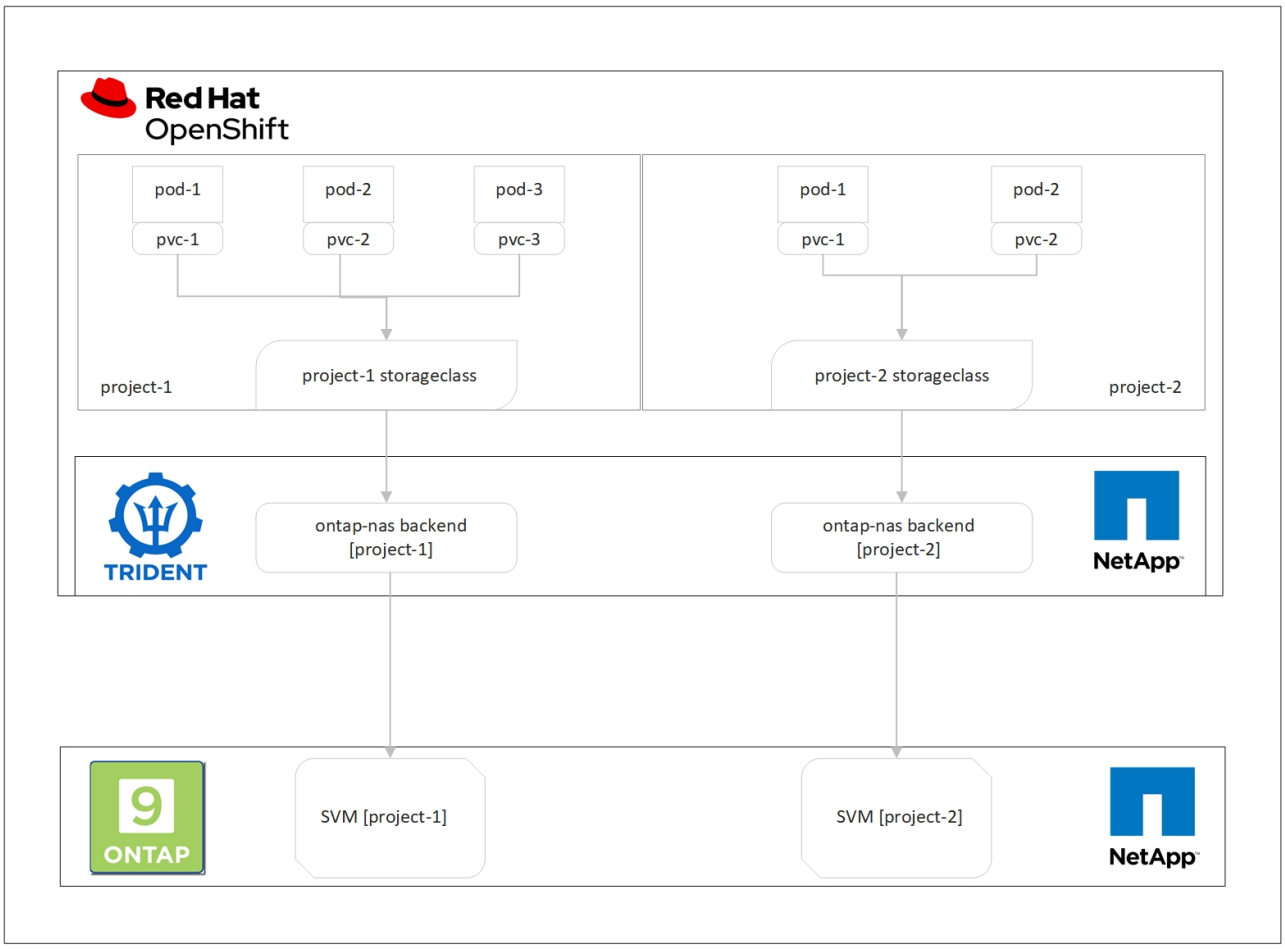
"次の例は、アーキテクチャです"

アーキテクチャ

ネットアップ ONTAP を基盤とする Red Hat OpenShift と Astra Trident は、デフォルトでワークロードを分離する機能を提供していませんが、マルチテナンシーの設定に使用できる幅広い機能を備えています。ネットアップ ONTAP を基盤とする Astra Trident を使用した Red Hat OpenShift クラスタでのマルチテナント解決策の設計について理解を深めるために、一連の要件を含む例を検討し、その構成について概説します。

2 つの異なるチームが取り組んでいる 2 つのプロジェクトの一環として、組織が Red Hat OpenShift クラスタ上で 2 つのワークロードを実行するとします。こうしたワークロードのデータは、 NetApp ONTAP NAS バックエンドの Astra Trident によって動的にプロビジョニングされる PVC 上に存在します。組織では、この 2 つのワークロードに対応するマルチテナント解決策を設計し、これらのプロジェクトに使用されるリソースを分離して、セキュリティとパフォーマンスを維持することが求められています。主に、これらのアプリケーションを提供するデータに重点が置かれています。

次の図は、ネットアップ ONTAP を基盤とする Astra Trident を使用した Red Hat OpenShift クラスタ上のマルチテナント解決策を示しています。



テクノロジ要件

1. NetApp ONTAP ストレージクラスタ
2. Red Hat OpenShift クラスタ
3. Astra Trident

Red Hat OpenShift –クラスタリソース

Red Hat OpenShift クラスタの観点からは、最初に最上位のリソースがプロジェクトです。OpenShift プロジェクトは、 OpenShift クラスタ全体を複数の仮想クラスタに分割するクラスタリソースと見なすことができます。したがって、プロジェクトレベルでの分離によって、マルチテナンシーの設定の基盤が提供されます。

次に、クラスタで RBAC を設定します。ベストプラクティスとして、すべての開発者が 1 つのプロジェクトまたはワークロードを担当し、アイデンティティプロバイダ（ IdP ）内の単一のユーザグループに設定することを推奨します。Red Hat OpenShift では、 IdP の統合とユーザグループの同期が可能なため、 IdP のユーザとグループをクラスタにインポートできるようになります。これにより、クラスタ管理者は、プロジェクト専用のクラスタリソースへのアクセスをそのプロジェクトに使用するユーザグループまたはグループに分離して、クラスタリソースへの不正アクセスを制限できます。Red Hat OpenShift への IdP の統合の詳細については、のドキュメントを参照してください "こちらをご覧ください"。

NetApp ONTAP

Red Hat OpenShift クラスタの永続的ストレージプロバイダとして機能している共有ストレージを分離し、各プロジェクト用にストレージ上に作成されたボリュームが、別々のストレージ上に作成されたものと同じようにホストに表示されるようにすることが重要です。そのためには、プロジェクトやワークロードに応じて Storage Virtual Machine （ SVM ）を NetApp ONTAP 上に作成し、各 SVM をワークロード専用にします。

Astra Trident

NetApp ONTAP で作成されたプロジェクトごとに異なる SVM が作成されたら、各 SVM を異なる Trident バックエンドにマッピングする必要があります。Trident のバックエンド構成は、 OpenShift クラスタリソースへの永続的ストレージの割り当てを促進します。また、マッピング先の SVM の詳細が必要です。これは、バックエンドのプロトコルドライバである必要があります。必要に応じて、ストレージでのボリュームのプロビジョニング方法を定義したり、ボリュームのサイズやアグリゲートの使用などを制限したりできます。Trident バックエンドの定義に関する詳細はこちらをご覧ください "こちらをご覧ください"。

Red Hat OpenShift –ストレージリソース

Trident バックエンドを設定したら、次の手順として StorageClasses を設定します。バックエンドと同じ数のストレージクラスを構成して、各ストレージクラスが 1 つのバックエンドにしかボリュームをスピンアップできない。ストレージクラスを定義する際に StoragePools パラメータを使用して、ストレージクラスを特定の Trident バックエンドにマッピングできます。ストレージクラスを定義する詳細については、を参照してください "こちらをご覧ください"。そのため、 StorageClass から Trident バックエンドへの 1 対 1 のマッピングで、 1 つの SVM をポイントします。これにより、そのプロジェクトに割り当てられた StorageClass を経由するすべてのストレージ要求が、そのプロジェクト専用の SVM によって処理されます。

ストレージクラスにネームスペースリソースが含まれていないため、あるプロジェクトのストレージクラスに対するストレージ要求を別のネームスペースまたはプロジェクトのポッドで拒否するにはどうすればよいですか？回答では、 ResourceQuotas を使用します。ResourceQuotas は、プロジェクトごとのリソースの合計使用量を制御するオブジェクトです。プロジェクト内のオブジェクトで消費できるリソースの合計量だけでなく、リソースの数も制限できます。ほとんどの場合、 ResourceQuotas を使用してプロジェクトのリソースを制限することができます。この機能を効率的に使用することで、リソースのオーバープロビジョニングや過剰消費によるコストやシステム停止を削減できます。のドキュメントを参照してください "こちらをご覧ください" を参照してください。

このユースケースでは、特定のプロジェクトのポッドが、プロジェクト専用ではないストレージクラスのストレージを要求しないように制限する必要があります。これを行うには '<storage-class-name>.storageeclass.storage0.k8sio/persistentvolumeclaims'0 を設定して ' 他のストレージ・クラスに対する永続的ボリューム要求を制限する必要がありますさらに、クラスタ管理者は、プロジェクト内の開発者が ResourceQuotas を変更するためのアクセス権を持っていないことを確認する必要があります。

"次へ：設定"

設定

マルチテナント解決策では、必要以上に多くのクラスタリソースにアクセスすることはできません。つまり、マルチテナンシー構成の一部として構成するリソースセット全体が、クラスタ管理者、ストレージ管理者、および各プロジェクトに取り組む開発者に分けられます。

次の表に、各ユーザが実行する各タスクを示します。

ロール	タスク
* Cluster-admin*	さまざまなアプリケーションやワークロード用のプロジェクトを作成できます
	Storage Admin 用の ClusterRoles および RoleBindings を作成します
	ロールとロールの作成特定のアクセス権を割り当てる開発者のためのバインド プロジェクト
	[オプション] 特定のノードでポッドをスケジュールするようにプロジェクトを設定します
* ストレージ管理者 *	NetApp ONTAP に SVM を作成する
	Trident バックエンドを作成
	ストレージクラスを作成します
	ストレージリソースクォータを作成します
* 開発者 *	割り当てられたプロジェクトで PVC またはポッドを作成またはパッチするためのアクセスを検証します
	アクセスを検証して、別のプロジェクトで PVC またはポッドを作成またはパッチします
	アクセス権を検証して、プロジェクト、リソースクォータ、ストレージクラスを表示または編集します

"次の手順：前提条件"

設定

前提条件

- NetApp ONTAP クラスタ：
- Red Hat OpenShift クラスタ
- Trident がクラスタにインストールされている。
- tridentctl および OC ツールがインストールされ、 $PATH に追加された管理ワークステーション。
- ONTAP への管理アクセス。
- OpenShift クラスタへのクラスタ管理者アクセス。
- クラスタがアイデンティティプロバイダに統合されました。
- アイデンティティプロバイダは、異なるチームのユーザを効率的に区別するように設定されています。

"次：クラスタ管理者のタスク"

Configuration ：クラスタ管理者のタスク

Red Hat OpenShift cluster-admin によって次のタスクが実行されます。

1. Red Hat OpenShift クラスタに cluster-admin としてログインします。

2. 異なるプロジェクトに対応する 2 つのプロジェクトを作成します。

```
oc create namespace project-1
oc create namespace project-2
```

3. project-1 の開発者ロールを作成します。

```
cat << EOF | oc create -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: project-1
  name: developer-project-1
rules:
  - verbs:
      - '*'
    apiGroups:
      - apps
      - batch
      - autoscaling
      - extensions
      - networking.k8s.io
      - policy
      - apps.openshift.io
      - build.openshift.io
      - image.openshift.io
      - ingress.operator.openshift.io
      - route.openshift.io
      - snapshot.storage.k8s.io
      - template.openshift.io
    resources:
      - '*'
  - verbs:
      - '*'
    apiGroups:
      - ''
    resources:
      - bindings
      - configmaps
      - endpoints
      - events
      - persistentvolumeclaims
      - pods
      - pods/log
      - pods/attach
```

```
      - podtemplates
      - replicationcontrollers
      - services
      - limitranges
      - namespaces
      - componentstatuses
      - nodes
  - verbs:
      - '*'
    apiGroups:
      - trident.netapp.io
    resources:
      - tridentsnapshots
EOF
```

ここで説明するロール定義は単なる例です。エンドユーザの要件に基づいて開発者の役割を定義する必要があります。

1. 同様に、 project-2 の開発者ロールを作成します。
2. すべての OpenShift およびネットアップストレージリソースは、通常はストレージ管理者が管理します。ストレージ管理者向けのアクセスは、 Trident のインストール時に作成された Trident オペレータロールによって制御されます。これに加えて、ストレージ管理者は ResourceQuotas にアクセスして、ストレージの消費方法を制御する必要があります。
3. クラスタ内のすべてのプロジェクトの ResourceQuotas を管理する役割を作成して、ストレージ管理者に割り当てます。

```
cat << EOF | oc create -f -
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: resource-quotas-role
rules:
  - verbs:
      - '*'
    apiGroups:
      - ''
    resources:
      - resourcequotas
  - verbs:
      - '*'
    apiGroups:
      - quota.openshift.io
    resources:
      - '*'
EOF
```

4. クラスタが組織のアイデンティティプロバイダと統合され、ユーザグループがクラスタグループと同期されていることを確認します。次の例は、アイデンティティプロバイダがクラスタに統合され、ユーザグループと同期されていることを示しています。

```
$ oc get groups
NAME                        USERS
ocp-netapp-storage-admins   ocp-netapp-storage-admin
ocp-project-1               ocp-project-1-user
ocp-project-2               ocp-project-2-user
```

1. ストレージ管理者用の ClusterRoleBindings を設定します。

```
cat << EOF | oc create -f -
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: netapp-storage-admin-trident-operator
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-netapp-storage-admins
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: trident-operator
---
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: netapp-storage-admin-resource-quotas-cr
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-netapp-storage-admins
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: resource-quotas-role
EOF
```

ストレージ管理者の場合は、 Trident オペレータとリソースクォータの 2 つのロールにバインドする必要があります。

1. ロールの作成 - developer-project-1 のロールを project-1 の対応するグループ (OCP-project-1) にバインドする開発者のバインディング。

```
cat << EOF | oc create -f -
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: project-1-developer
  namespace: project-1
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-project-1
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: developer-project-1
EOF
```

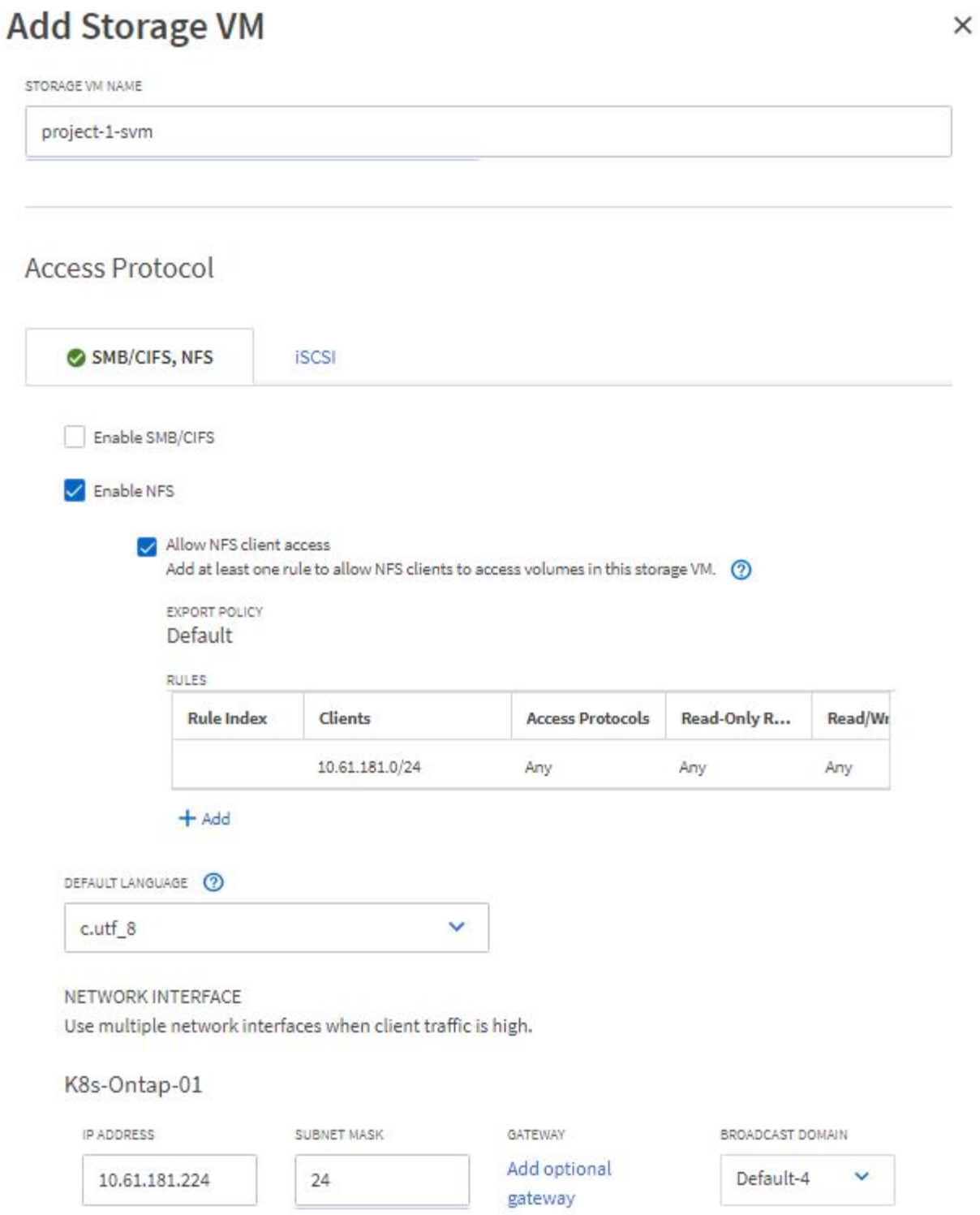
2. 同様に、開発者の役割を project-2 の対応するユーザーグループにバインドする開発者の RoleBindings を作成します。

"次：ストレージ管理者のタスク"

設定：ストレージ管理者のタスク

ストレージ管理者が次のリソースを設定する必要があります。

1. NetApp ONTAP クラスタに admin としてログインします。
2. Storage > Storage VMs と進み、 Add をクリックします。必要な詳細を指定して、プロジェクト 1 用とプロジェクト 2 用に 1 つずつ、 2 つの SVM を作成します。また、 SVM とそのリソースを管理するには vsadmin アカウントを作成します。



1. ストレージ管理者として Red Hat OpenShift クラスタにログインします。
2. project-1 のバックエンドを作成し、プロジェクト専用の SVM にマッピングします。ONTAP クラスタ管理者を使用する代わりに、 SVM の vsadmin アカウントを使用してバックエンドを SVM に接続することを推奨します。

```
cat << EOF | tridentctl -n trident create backend -f
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "backendName": "nfs_project_1",
    "managementLIF": "172.21.224.210",
    "dataLIF": "10.61.181.224",
    "svm": "project-1-svm",
    "username": "vsadmin",
    "password": "NetApp123"
}
EOF
```

この例では ONTAP と NAS のドライバを使用しています。ユースケースに基づいてバックエンドを作成する場合は、適切なドライバを使用します。

Trident が Trident プロジェクトにインストールされているとします。

1. 同様に、 project-2 の Trident バックエンドを作成し、 project-2 に専用の SVM にマッピングします。
2. 次に、ストレージクラスを作成します。StoragePools パラメータを設定して、 project-1 専用のバックエンドのストレージプールを使用するように project-1 のストレージクラスを作成し、これを設定します。

```
cat << EOF | oc create -f -
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: project-1-sc
provisioner: csi.trident.netapp.io
parameters:
  backendType: ontap-nas
  storagePools: "nfs_project_1:.*"
EOF
```

3. 同様に、 project-2 に対してストレージクラスを作成し、 project-2 に専用のバックエンドのストレージプールを使用するように設定します。
4. ResourceQuota を作成して ' プロジェクト 1 内のリソースを制限し ' 他のプロジェクト専用のストレージを要求します

```
cat << EOF | oc create -f -
kind: ResourceQuota
apiVersion: v1
metadata:
  name: project-1-sc-rq
  namespace: project-1
spec:
  hard:
    project-2-sc.storageclass.storage.k8s.io/persistentvolumeclaims: 0
EOF
```

5. 同様に 'ResourceQuota を作成して 'project-2 内のリソースを制限し ' 他のプロジェクト専用のストレージを要求します

"次のステップ：検証"

検証

前の手順で設定したマルチテナントアーキテクチャを検証するには、次の手順を実行します。

割り当てられたプロジェクトで **PVC** またはポッドを作成するためのアクセスを検証します

1. OCP-project-1-user として、 project-1 の開発者としてログインします。
2. アクセス権をチェックして新しいプロジェクトを作成してください

```
oc create ns sub-project-1
```

3. project-1 に割り当てられたストレージクラスを使用して 'project-1 に PVC を作成します

```
cat << EOF | oc create -f -
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: test-pvc-project-1
  namespace: project-1
  annotations:
    trident.netapp.io/reclaimPolicy: Retain
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: project-1-sc
EOF
```

4. PVC に関連付けられている PV を確認します

```
oc get pv
```

5. PV とそのボリュームが、 NetApp ONTAP 上のプロジェクト 1 専用の SVM に作成されていることを確認します。

```
volume show -vserver project-1-svm
```

6. project-1 にポッドを作成し、前の手順で作成した PVC をマウントします。

```
cat << EOF | oc create -f -
kind: Pod
apiVersion: v1
metadata:
  name: test-pvc-pod
  namespace: project-1
spec:
  volumes:
    - name: test-pvc-project-1
      persistentVolumeClaim:
       claimName: test-pvc-project-1
  containers:
    - name: test-container
      image: nginx
      ports:
        - containerPort: 80
          name: "http-server"
      volumeMounts:
        - mountPath: "/usr/share/nginx/html"
          name: test-pvc-project-1
EOF
```

7. ポッドが実行中かどうか、およびボリュームがマウントされているかどうかを確認します。

```
oc describe pods test-pvc-pod -n project-1
```

アクセスを検証して別のプロジェクトに **PVC** またはポッドを作成するか、別のプロジェクト専用のリソースを使用します

1. OCP-project-1-user として、 project-1 の開発者としてログインします。
2. project-2 に割り当てられたストレージクラスを使用して 'project-1 に PVC を作成します

```
cat << EOF | oc create -f -
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: test-pvc-project-1-sc-2
  namespace: project-1
  annotations:
    trident.netapp.io/reclaimPolicy: Retain
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: project-2-sc
EOF
```

3. PROJECT -2 で PVC を作成します。

```
cat << EOF | oc create -f -
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: test-pvc-project-2-sc-1
  namespace: project-2
  annotations:
    trident.netapp.io/reclaimPolicy: Retain
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: project-1-sc
EOF
```

4. PVC 「 test-pvc-project-1-sc-2 」および「 test-pvc-project-2-ssc-1 」が作成されていないことを確認します。

```
oc get pvc -n project-1
oc get pvc -n project-2
```

5. プロジェクト 2 でポッドを作成します。

```
cat << EOF | oc create -f -
kind: Pod
apiVersion: v1
metadata:
  name: test-pvc-pod
  namespace: project-1
spec:
  containers:
    - name: test-container
      image: nginx
      ports:
        - containerPort: 80
          name: "http-server"
EOF
```

アクセス権を検証して、プロジェクト、リソースクォータ、ストレージクラスを表示および編集します

1. OCP-project-1-user として、 project-1 の開発者としてログインします。
2. アクセス権をチェックして新しいプロジェクトを作成してください。

```
oc create ns sub-project-1
```

3. アクセスを検証してプロジェクトを表示します

```
oc get ns
```

4. ユーザーがで ResourceQuotas を表示または編集できるかどうかを確認します プロジェクト 1

```
oc get resourcequotas -n project-1
oc edit resourcequotas project-1-sc-rq -n project-1
```

5. ユーザーがストレージクラスを表示するためのアクセス権を持っていることを確認します

```
oc get sc
```

6. ストレージクラスについては ' アクセスを確認してください
7. ストレージクラスを編集するためにユーザーのアクセス権を検証します

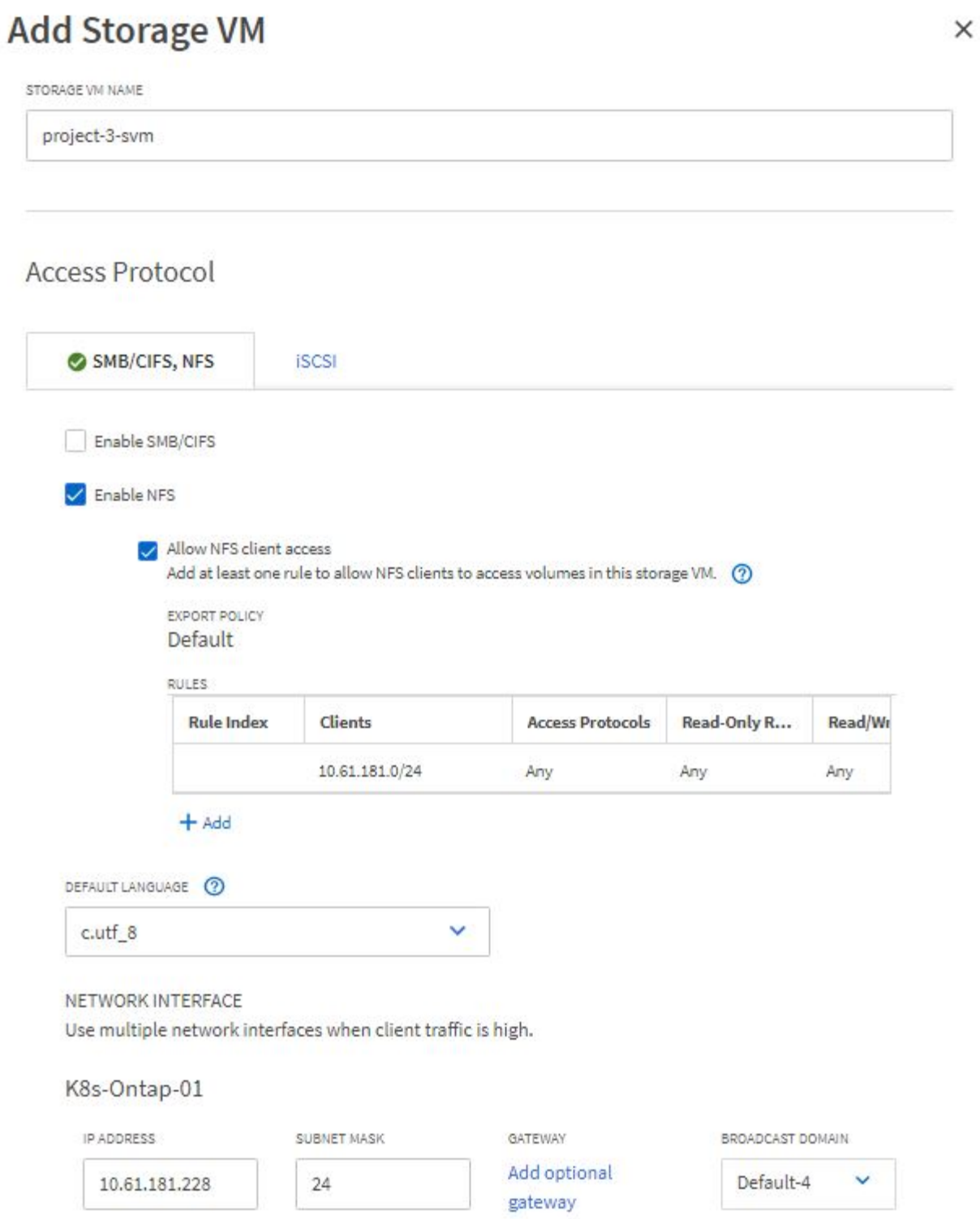
```
oc edit sc project-1-sc
```

"次のステップ：スケーリング"

拡張：プロジェクトの追加

マルチテナント構成でストレージリソースを使用する新しいプロジェクトを追加する場合、マルチテナンシーを違反しないように追加の設定が必要になります。マルチテナントクラスタでプロジェクトを追加するには、次の手順を実行します。

1. NetApp ONTAP クラスタにストレージ管理者としてログインします。
2. 「ストレージ → ストレージ VM 」に移動し、「追加」をクリックします。project-3 専用の新しい SVM を作成します。また、 SVM とそのリソースを管理するには vsadmin アカウントを作成します。



1. Red Hat OpenShift クラスタにクラスタ管理者としてログインします
2. 新しいプロジェクトを作成します。

```
oc create ns project-3
```

3. IdP に project-3 のユーザグループが作成され、 OpenShift クラスタと同期されていることを確認してください。

```
oc get groups
```

4. project-3 の開発者ロールを作成します。

```
cat << EOF | oc create -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: project-3
  name: developer-project-3
rules:
  - verbs:
      - '*'
    apiGroups:
      - apps
      - batch
      - autoscaling
      - extensions
      - networking.k8s.io
      - policy
      - apps.openshift.io
      - build.openshift.io
      - image.openshift.io
      - ingress.operator.openshift.io
      - route.openshift.io
      - snapshot.storage.k8s.io
      - template.openshift.io
    resources:
      - '*'
  - verbs:
      - '*'
    apiGroups:
      - ''
    resources:
      - bindings
      - configmaps
      - endpoints
      - events
      - persistentvolumeclaims
      - pods
      - pods/log
      - pods/attach
```

```
      - podtemplates
      - replicationcontrollers
      - services
      - limitranges
      - namespaces
      - componentstatuses
      - nodes
  - verbs:
      - '*'
    apiGroups:
      - trident.netapp.io
    resources:
      - tridentsnapshots
EOF
```

ここで説明するロール定義は単なる例です。開発者ロールは、エンドユーザの要件に基づいて定義する必要があります。

1. プロジェクト 3 の開発者用に RoleBinding を作成します。これは、 developer-project-3 の役割を、project-3 の対応するグループ (OCP-project-3) にバインドします。

```
cat << EOF | oc create -f -
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: project-3-developer
  namespace: project-3
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-project-3
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: developer-project-3
EOF
```

2. Red Hat OpenShift クラスタにストレージ管理者としてログインします
3. Trident バックエンドを作成し、 project-3 専用の SVM にマッピングします。ONTAP クラスタ管理者を使用する代わりに、 SVM の vsadmin アカウントを使用してバックエンドを SVM に接続することを推奨します。

```
cat << EOF | tridentctl -n trident create backend -f
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "backendName": "nfs_project_3",
    "managementLIF": "172.21.224.210",
    "dataLIF": "10.61.181.228",
    "svm": "project-3-svm",
    "username": "vsadmin",
    "password": "NetApp!23"
}
EOF
```

この例では ONTAP と NAS のドライバを使用しています。ユースケースに基づいてバックエンドを作成するための適切なドライバを使用します。

Trident が Trident プロジェクトにインストールされているとします。

1. project-3 用のストレージクラスを作成し、 project-3 専用のバックエンドのストレージプールを使用するように設定します。

```
cat << EOF | oc create -f -
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: project-3-sc
provisioner: csi.trident.netapp.io
parameters:
  backendType: ontap-nas
  storagePools: "nfs_project_3:.*"
EOF
```

2. ResourceQuota を作成して ' プロジェクト 3 のリソースを制限しますストレージを要求するストレージは ' 他のプロジェクト専用のストレージになります

```
cat << EOF | oc create -f -
kind: ResourceQuota
apiVersion: v1
metadata:
  name: project-3-sc-rq
  namespace: project-3
spec:
  hard:
    project-1-sc.storageclass.storage.k8s.io/persistentvolumeclaims: 0
    project-2-sc.storageclass.storage.k8s.io/persistentvolumeclaims: 0
EOF
```

3. 他のプロジェクトの ResourceQuotas にパッチを適用して ' プロジェクト内のリソースがプロジェクト 3 専用のストレージからストレージにアクセスするのを制限します

```
oc patch resourcequotas project-1-sc-rq -n project-1 --patch
'{"spec":{"hard":{ "project-3-
sc.storageclass.storage.k8s.io/persistentvolumeclaims": 0}}}'
oc patch resourcequotas project-2-sc-rq -n project-2 --patch
'{"spec":{"hard":{ "project-3-
sc.storageclass.storage.k8s.io/persistentvolumeclaims": 0}}}'
```

著作権情報

Copyrightゥ2022 NetApp、Inc. All rights reserved.米国で印刷されていますこのドキュメントは著作権によって保護されています。画像媒体、電子媒体、および写真複写、記録媒体などの機械媒体など、いかなる形式および方法による複製も禁止します。 テープ媒体、または電子検索システムへの保管-著作権所有者の書面による事前承諾なし。

ネットアップの著作物から派生したソフトウェアは、次に示す使用許諾条項および免責条項の対象となります。

このソフトウェアは、ネットアップによって「現状のまま」提供されています。ネットアップは明示的、または商品性および特定目的に対する適合性の暗示的保証を含み、かつこれに限定されないいかなる暗示的な保証も行いません。ネットアップは、いかなる場合でも、間接的、偶発的、特別、懲罰的、または または結果的損害（代替品または代替サービスの調達、使用の損失、データ、利益、またはこれらに限定されないものを含みますが、これらに限定されません。） ただし、契約、厳格責任、または本ソフトウェアの使用に起因する不法行為（過失やその他を含む）のいずれであっても、かかる損害の可能性について知らされていた場合でも、責任の理論に基づいて発生します。

ネットアップは、ここに記載されているすべての製品に対する変更を随時、予告なく行う権利を保有します。ネットアップによる明示的な書面による合意がある場合を除き、ここに記載されている製品の使用により生じる責任および義務に対して、ネットアップは責任を負いません。この製品の使用または購入は、ネットアップの特許権、商標権、またはその他の知的所有権に基づくライセンスの供与とはみなされません。

このマニュアルに記載されている製品は、 1 つ以上の米国特許、その他の国の特許、および出願中の特許によ特許、その他の国の特許、および出願中の特許。

権利の制限について：政府による使用、複製、開示は、 DFARS 252.227-7103 （ 1988 年 10 月）および FAR 52-227-19 （ 1987 年 6 月）の Rights in Technical Data and Computer Software （技術データおよびコンピュータソフトウェアに関する諸権利）条項の（ c ）（ 1 ）（ ii ）項、に規定された制限が適用されます。

商標情報

NetApp、NetAppのロゴ、に記載されているマーク http://www.netapp.com/TM は、NetApp、Inc.の商標です。その他の会社名と製品名は、それを所有する各社の商標である場合があります。